SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1010	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	S.BHAVYA	**Reg. No.:**	192524376
Date:	29.10.26	**Duration:**	60 minutes

Code of Conduct

I Bhavya S(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Bhavya S*

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
   - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
Problem Analysis & Project Design (20%)	/5
Git Repository & Branching Strategy(20%)	/4
Version Control Workflow & Commit History(10%)	/4
Docker Implementation(20%)	/4
Deployment, Testing & Results(15%)	/4
Documentation, Challenges & Future Enhancements(15%)	/4
TOTAL	/25

PERSONAL EXPENSE AND BUDGET MANAGEMENT SYSTEM

1. Problem Analysis

1.1 Problem Statement

The Personal Expense and Budget Management System is a web application that allows users to record income and expenses, categorize spending, create budgets, receive alerts, and view financial reports. It helps users manage their finances efficiently, reduce unnecessary spending, and monitor their savings.

Manual financial tracking can cause errors, missed transactions, and overspending. The proposed system provides a centralized platform for maintaining financial records and monitoring budgets.

1.2 Objectives

1. To record and manage income and expenses.
2. To categorize expenses for easy tracking.
3. To create monthly budgets and monitor spending limits.
4. To generate alerts when spending approaches or exceeds a budget.
5. To display financial summaries and reports.
6. To maintain financial records securely and accurately.
7. To use Git for version control and Docker for consistent application deployment.

1.3 Functional Requirements

- User Management: Register and log in to the application.
- Income Management: Add, update, and view income records.
- Expense Management: Record expenses with amounts, dates, and categories.
- Budget Management: Set monthly and category-wise spending limits.
- Alert Management: Notify users when spending reaches a configured threshold.
- Financial Reports: Display total income, expenses, savings, and category-wise spending.

1.4 Non-Functional Requirements

Requirement	Description
Security	Protect user accounts and financial records.
Performance	Retrieve transactions and reports efficiently.
Reliability	Maintain accurate financial data.
Scalability	Support increasing users and transactions.
Maintainability	Allow modules to be updated easily.
Portability	Run the application consistently using Docker.

1.5 Expected Outcomes

The completed application should allow users to maintain income and expense records, monitor their budgets, receive spending alerts, and view financial reports. Git should maintain the project's development history, while Docker should simplify deployment and provide a consistent execution environment.

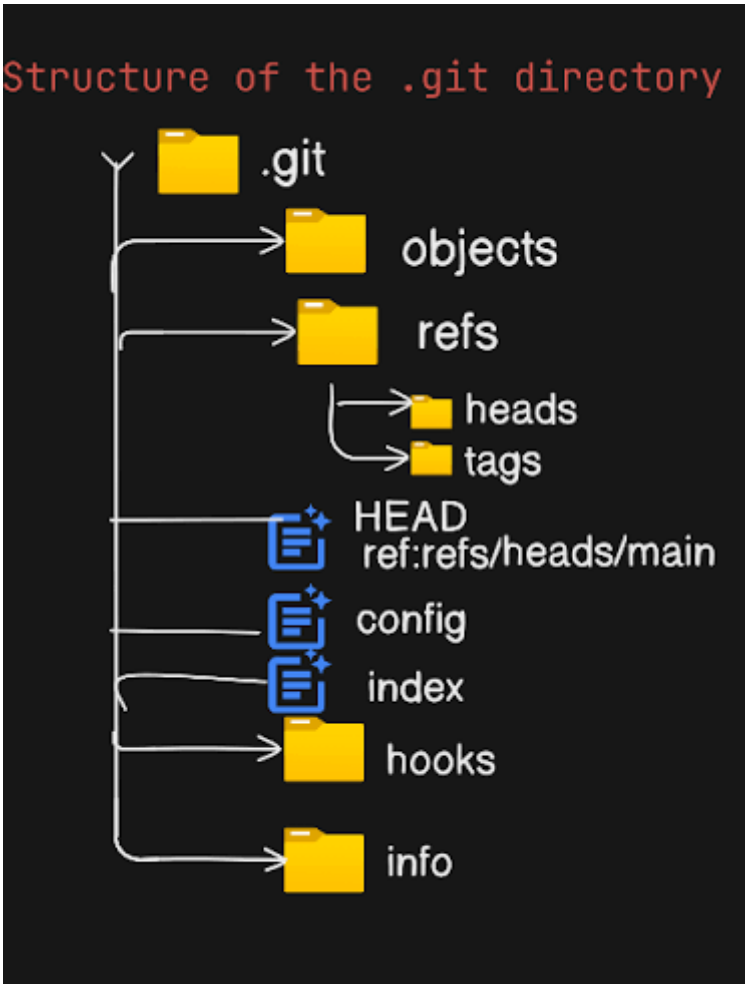
2. Project Structure Design

2.1 Proposed Folder Structure

The project repository is organized into the following folders and files.

personal-expense-budget-system/

```
│
├── app.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .gitignore
├── .env.example
├── README.md
│
├── templates/
│   ├── index.html
│   ├── expenses.html
│   ├── budget.html
│   └── reports.html
│
├── static/
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── script.js
│
├── database/
│   └── init.sql
│
├── tests/
│   └── test_app.py
│
└── docs/
    ├── architecture.png
    └── screenshots/
```



2.2 Purpose of Each File and Folder

File or folder	Purpose
app.py	Contains Flask routes and application logic.
requirements.txt	Lists Python dependencies.
Dockerfile	Defines how to build the application image.
docker-compose.yml	Configures the application and database services.
.gitignore	Excludes unnecessary and sensitive files from Git.
.env.example	Documents required environment variables without real secrets.
README.md	Contains project details, setup instructions, and usage.
templates/	Stores HTML pages displayed to users.
static/css/	Stores the application's styling.
static/js/	Stores browser-side JavaScript.
database/init.sql	Defines the initial database schema.
tests/	Contains automated tests.
docs/	Stores architecture diagrams and screenshots.

2.3 Justification

This structure separates application logic, user-interface files, database setup, tests, and documentation. It makes the repository easier to understand and maintain.

The Dockerfile and Docker Compose configuration support containerization, while .gitignore prevents unnecessary files and local secrets from entering version control. The tests/ folder supports verification of application functionality.

The structure also allows multiple developers to work on separate components, such as the interface, backend, database, and testing.

3. Git Repository Initialization

3.1 Introduction

Git is a distributed version control system that tracks changes in source code. It helps developers maintain project history, create branches, collaborate, and restore earlier versions when required.

GitHub can be used to host the repository and share the project with team members.

3.2 Steps to Initialize the Repository

Step 1: Create the project folder
Open Git Bash or Command Prompt and execute:

```
mkdir personal-expense-budget-system
cd personal-expense-budget-system
```

Step 2: Initialize Git

```
git init
```

This initializes a Git repository in the project folder.
Step 3: Create project files
Create the application, documentation, database, and Docker configuration files using the folder structure in Question 2.
Step 4: Configure Git identity

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Replace these values with your own Git identity.
Step 5: Add files and commit

```
git add .
git commit -m "Initial project structure"
```

The git add command stages files, and git commit records the staged changes.
Step 6: Connect to GitHub
Create an empty GitHub repository named personal-expense-budget-system, then connect it using the repository URL provided by GitHub.

```
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Replace YOUR_GITHUB_REPOSITORY_URL with the actual URL of your repository.

3.3 Essential Git Files and Commands

Item	Purpose
.git/	Stores repository history and Git metadata.
.gitignore	Specifies files Git should ignore.
git status	Shows modified, staged, and untracked files.
git add	Stages changes for a commit.
git commit	Saves a snapshot of staged changes.
git log	Displays commit history.
git remote	Manages remote repository connections.
git push	Uploads local commits to the remote repository.

3.4 Sample .gitignore

__pycache__/

*.pyc

.env

.venv/

venv/

.pytest_cache/

This file prevents Python cache files, virtual environments, and local environment files from being tracked. Real credentials should never be committed.

3.5 Result

The Git repository is initialized and organized. The initial project structure is committed and can be synchronized with GitHub for version control and collaboration.

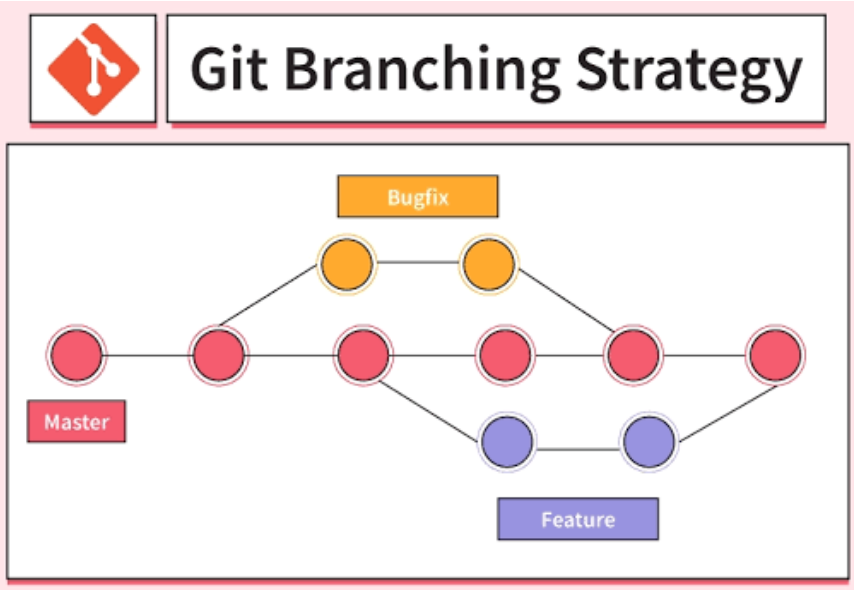
4. Git Branching Strategy

4.1 Selected Strategy

The project uses a lightweight feature-branch workflow with the following branches:

- Main: Contains stable and tested code.
- Development: Integrates completed features before release.
- Feature branches: Used to develop individual functions.
- Hotfix branches: Used to correct urgent defects in the stable version.

4.2 Branch Structure Diagram



4.3 Branch Creation Commands

Create the development branch:

```
git branch development
git switch development
```

Create a feature branch:

```
git switch -c feature/expenses
```

After completing the expense module, stage and commit its changes:

```
git add .
git commit -m "Add expense management module"
```

Return to development and merge the feature:

```
git switch development
git merge feature/expenses
```

After testing, the development branch can be merged into main:

```
git switch main
git merge development
git push origin main
```

4.4 Justification

This branching strategy is suitable because different project features can be developed independently. The main branch remains stable while development and feature branches allow new functionality to be implemented and tested.

It reduces the risk of unfinished code affecting the stable application. It also supports code review and collaborative development through GitHub pull requests.

For a small student project, this workflow is simpler than a full GitFlow implementation while retaining the important separation between feature development, integration, and stable releases.

5. Version Control Workflow

5.1 Git Workflow

The version control workflow includes creating branches, modifying files, committing changes, merging branches, resolving conflicts, and synchronizing with GitHub.

Step 1: Check Repository Status

```
git status
```

This command identifies changed and untracked files.

Step 2: Create a Feature Branch

```
git switch development
git switch -c feature/budget
```

This creates an isolated branch for budget functionality.

Step 3: Modify the Application

Implement budget creation, spending calculations, and threshold checks. After modifying the files, review the changes:

```
git diff
```

Step 4: Commit the Changes

```
git add app.py
git commit -m "Add budget tracking and threshold checks"
```

A meaningful commit message explains what was changed.

Step 5: Merge the Feature

```
git switch development
git merge feature/budget
```

Git integrates the changes into the development branch.

Step 6: Synchronize with GitHub

```
git push -u origin development
```

This uploads the development branch to the remote repository.

To retrieve the latest remote changes:

```
git pull origin development
```

Step 7: Release the Tested Application

After testing the integrated application:

```
git switch main
git merge development
git push origin main
```

This updates the stable branch with the tested development changes.

5.2 Conflict Resolution

A merge conflict may occur when two branches modify the same part of a file.

To resolve it:

1. Run git status to identify conflicting files.
2. Open each file and locate the conflict markers.
3. Edit the file to retain the correct combined implementation.
4. Stage the resolved file.
5. Complete the merge commit.

Commands:

```
git status
git add app.py
git commit -m "Resolve merge conflict in application"
```

The final commit command applies when Git has left the merge in progress and requires a commit. If Git completes a fast-forward merge or another automatic merge, an additional commit may not be necessary.

5.3 Pull Request Workflow

When using GitHub, a developer can push a feature branch and create a pull request targeting development. Another developer can review the code and request corrections before merging it.

This workflow improves collaboration, code quality, and traceability.

5.4 Importance of Git Operations

Operation	Purpose
git init	Creates the local repository.
git add	Stages changes.
git commit	Records a version.
git branch	Creates or lists branches.
git merge	Integrates branch changes.
git pull	Retrieves and integrates remote changes.
git push	Uploads commits to the remote repository.
git log	Displays project history.

6. Commit History Analysis

6.1 Importance of Commit History

Commit history records the sequence of changes made to the project. It helps developers identify when a feature was added, understand earlier modifications, and investigate defects.

Each commit should contain a clear message describing the change. Small, focused commits are easier to review and maintain than large commits containing unrelated modifications.

6.2 Sample Commit History

The following is an example of the expected history after implementing the project. These are illustrative entries, not actual repository output.

a81c204 Add Docker Compose configuration
b62d913 Create Dockerfile for Flask application
c43e712 Add budget alerts and reports
d24f601 Add income and expense management
e15a390 Create initial project structure

6.3 Good Commit Message Practices

Examples of appropriate commit messages include:

- Add income and expense management
- Implement monthly budget calculation
- Add spending threshold alerts
- Create MySQL database schema
- Containerize Flask application
- Configure application and database services
- Add tests for budget calculations
- Fix expense validation error

Commit messages should be concise, specific, and written in a consistent style.

6.4 Viewing Commit History

Execute:

```
git log --oneline
```

To view changes introduced by a particular commit:

```
git show COMMIT_ID
```

Replace COMMIT_ID with an actual commit identifier.

6.5 Result

A meaningful commit history makes the project easier to maintain, supports collaboration, and helps developers trace changes during debugging and future development.

7. Docker Environment Design

7.1 Introduction

Docker is a containerization platform that packages an application and its dependencies into portable containers. Containers provide a consistent environment for running the application on different systems.

Docker is suitable for the Personal Expense and Budget Management System because the Flask application and MySQL database can run as separate services and communicate through a Docker network.

7.2 Components Requiring Containerization

The project uses two main containers.

1. Flask Application Container

This container runs the backend application, handles user requests, performs expense and budget calculations, and generates reports.

2. MySQL Database Container

This container stores user details, income, expenses, budgets, and related information.

Docker Compose manages both containers and configures their communication.

7.3 Docker Architecture Diagram

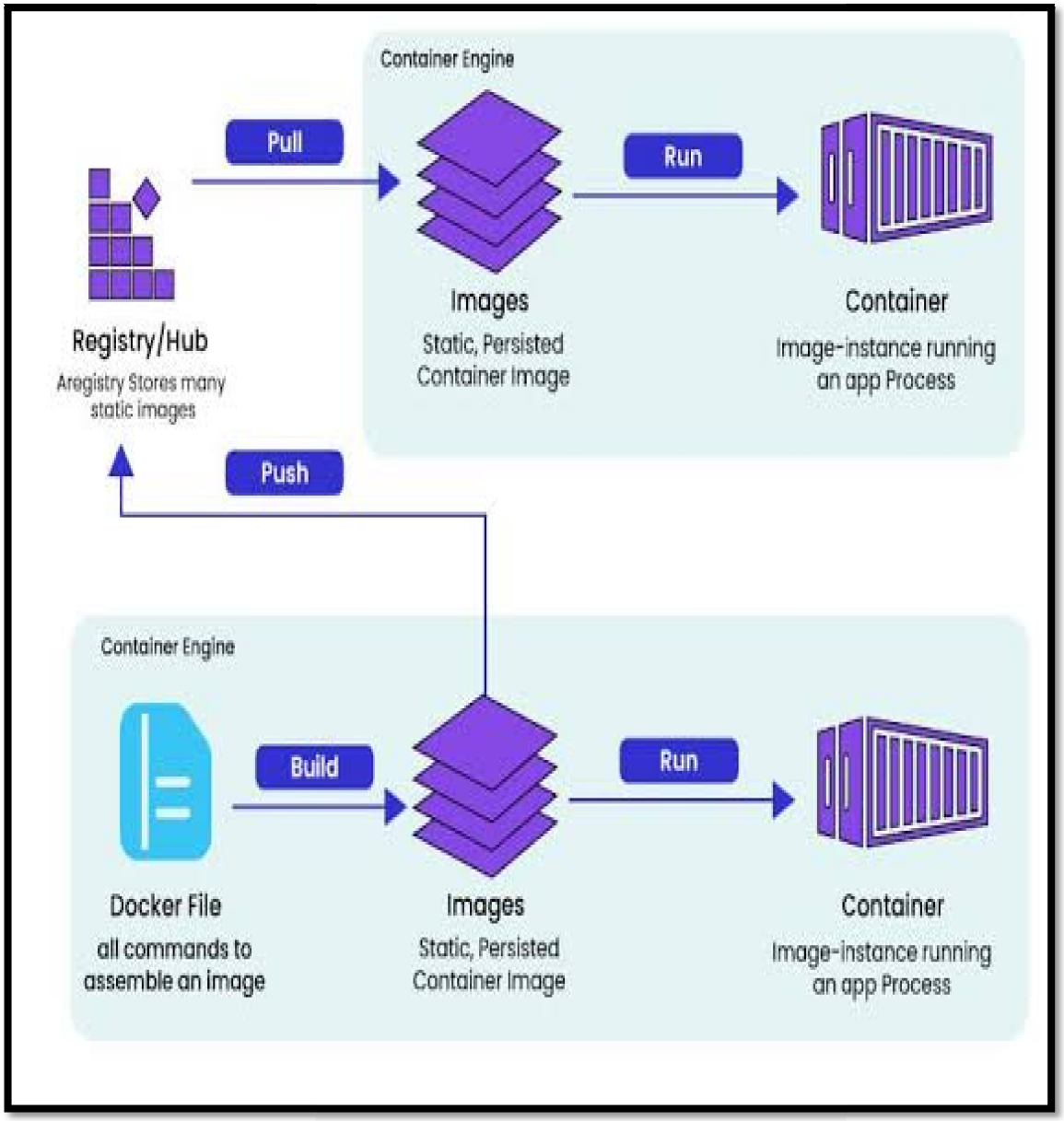
User / Web Browser
Accesses the application through port 5000
Docker Compose Environment
Flask Application Container
Python • Flask • Expense and Budget Logic
Internal Docker Network • Port 3306
MySQL Database Container
Users • Transactions • Budgets • Reports
Persistent Database Volume
Preserves database files when the container is recreated

7.4 Why Docker Is Suitable

- Portability: The same container image can run on different compatible systems.
- Consistency: Dependencies are packaged with the application.
- Isolation: The application and database run in separate containers.
- Easy Deployment: Docker Compose starts the required services together.
- Maintainability: Containers can be updated or recreated independently.
- Scalability: Application instances can be increased when needed, subject to database and application design.

Docker simplifies deployment, although security, persistent storage, backups, and application configuration must still be managed correctly.

8. Dockerfile Development



8.1 Purpose of the Dockerfile

A Dockerfile contains instructions for building the Flask application image. It defines the base image, working directory, dependencies, source files, listening port, and application startup command.

Create a file named Dockerfile in the project's root folder.

8.2 Dockerfile Code

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPYrequirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

8.3 Explanation of Dockerfile Instructions

Instruction	Purpose
FROM	Selects a lightweight Python base image.
WORKDIR	Sets the working directory inside the container.
ENV	Configures Python execution behavior.
COPY	Copies project files into the image.
RUN	Installs the required Python packages.
EXPOSE	Documents the application's listening port.
CMD	Starts the Flask application when the container runs.

The dependency file is copied before the rest of the source code so that Docker can reuse the dependency-installation layer when application code changes but the dependencies remain the same.

8.4 Requirements File

Create requirements.txt:

Flask

mysql-connector-python

These packages provide the Flask web framework and the MySQL database connector.

For a reproducible production deployment, use tested and pinned dependency versions.

8.5 Application Configuration

The Flask application should read its database configuration from environment variables rather than hardcoding database credentials.

For example:

```
import os
DB_HOST = os.getenv("DB_HOST", "db")
DB_NAME = os.getenv("DB_NAME", "expense_db")
DB_USER = os.getenv("DB_USER", "expense_user")
DB_PASSWORD = os.getenv("DB_PASSWORD")
```

The database hostname db refers to the database service name defined in Docker Compose.

8.6 Building the Docker Image

Run the following command from the project root:

```
docker build -t expense-budget-app .
```

Check that the image was created:

```
docker images
```

The generated image contains the application and its required Python dependencies.

9. Docker Compose Design

9.1 Introduction

Docker Compose is used to define and run multiple related containers through one configuration file. For this project, it manages the Flask application and MySQL database.

Create a file named docker-compose.yml in the project root.

9.2 Docker Compose Configuration

```
services:
  app:
    build: .
container_name: expense-budget-app
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_NAME: expense_db
      DB_USER: expense_user
      DB_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
depends_on:
db:
        condition: service_healthy
    restart: unless-stopped


db:
    image: mysql:8.4
container_name: expense-budget-db
    environment:
      MYSQL_DATABASE: expense_db
      MYSQL_USER: expense_user
      MYSQL_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?Set DB_ROOT_PASSWORD}
    volumes:
      - expense_data:/var/lib/mysql
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
healthcheck:
      test:
        - CMD-SHELL
        - "mysqladmin ping -h localhost -u root -p\"$${MYSQL_ROOT_PASSWORD}\""
      interval: 10s
      timeout: 5s
      retries: 10
    restart: unless-stopped

volumes:
expense_data:
```

9.3 Environment Variables

Create a local .env file in the project root containing:

DB_PASSWORD=ChangeThisAppPassword123

DB_ROOT_PASSWORD=ChangeThisRootPassword123

These are example values. Replace them with strong local passwords. Do not commit .env to GitHub.

The Compose configuration uses environment variables to supply database credentials and connection settings without embedding them directly in the application source code.

9.4 Services

Application Service: Builds the Flask image from the Dockerfile and publishes port 5000 to the host machine.

Database Service: Uses the MySQL image to store financial records and initializes the database with the supplied SQL file.

9.5 Networking

Docker Compose creates a default network for the services. The Flask application connects to MySQL using the hostname db and the internal port 3306.

Only the Flask application's port is published to the host. MySQL's port is not published in this configuration, which reduces unnecessary external exposure.

9.6 Volumes

The named volume expense_data stores MySQL data outside the database container's writable layer. This helps preserve database records when the container is recreated.

The SQL initialization file is mounted read-only into the database initialization directory. MySQL's standard image processes such initialization scripts when creating a fresh database directory.

Removing the named volume can permanently delete the stored database data. Backups should therefore be taken before destructive cleanup operations.

9.7 Dependencies and Health Checks

The application depends on the database service being healthy before Compose starts the application service.

The database health check verifies that MySQL responds to its local ping command. This helps prevent the application from starting before the database is ready, although the application should still handle temporary connection failures.

9.8 Database Initialization File

Create database/init.sql:

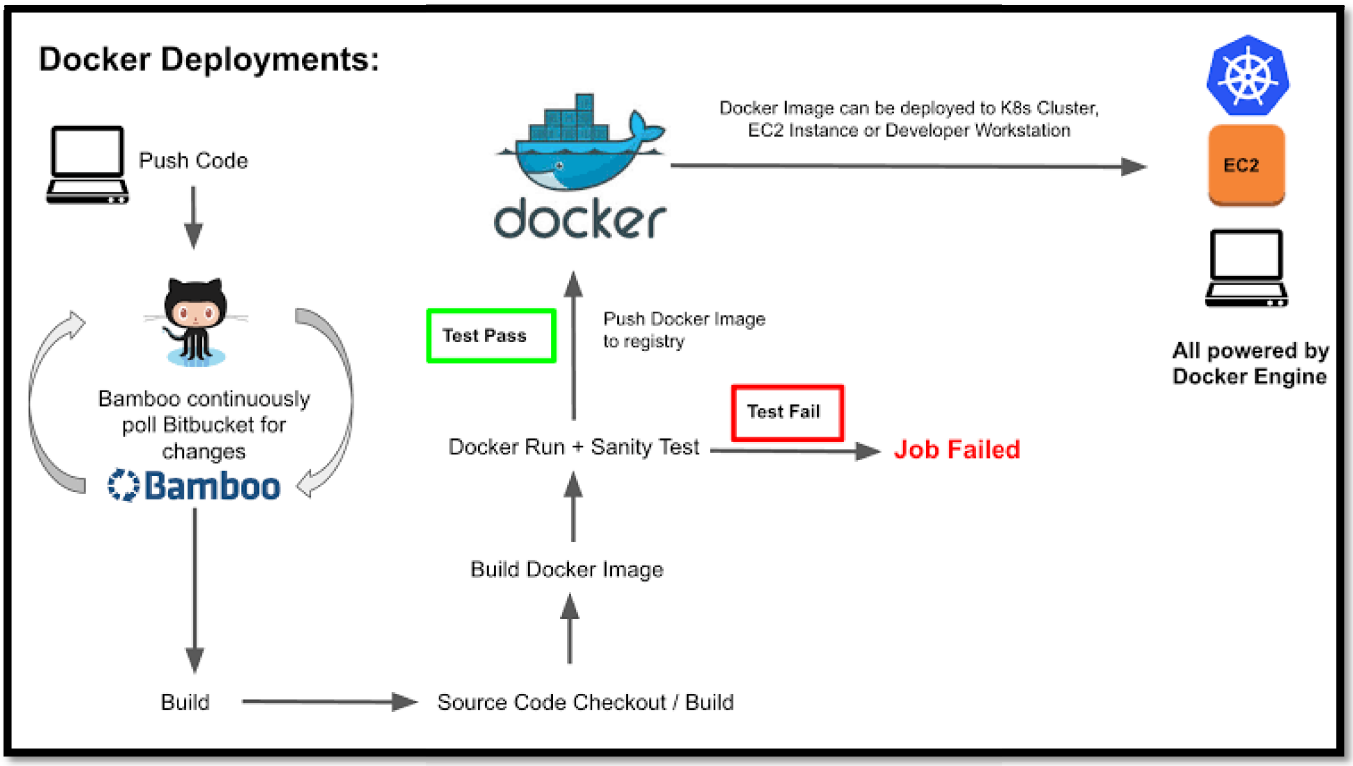
```
CREATE TABLE IF NOT EXISTS users (
user_id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(150) UNIQUE NOT NULL,
password_hashVARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS expenses (
expense_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,    amount DECIMAL(10,2) NOT NULL,
   category VARCHAR(100) NOT NULL,expense_date DATE NOT NULL,
   description VARCHAR(255),    FOREIGN KEY (user_id) REFERENCES users(user_id));
CREATE TABLE IF NOT EXISTS income (
income_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,    amount DECIMAL(10,2) NOT NULL,
income_date DATE NOT NULL,    source VARCHAR(100),
   FOREIGN KEY (user_id) REFERENCES users(user_id)
);
CREATE TABLE IF NOT EXISTS budgets (
budget_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
   category VARCHAR(100),
budget_limitDECIMAL(10,2) NOT NULL,
   period VARCHAR(20) NOT NULL,
   FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

This SQL file defines the basic database structure. Additional tables and constraints can be introduced for categories, notifications, and detailed reporting.

Important: The initialization script runs automatically only when MySQL initializes a fresh data directory. If the database volume already exists, modifying init.sql does not automatically recreate the tables.

10. Container Deployment and Testing



10.1 Deployment Procedure

The application is deployed using Docker Compose, which builds the Flask image and starts the application and MySQL services.

Step 1: Install prerequisites

Install Docker Desktop and ensure that its Docker Engine is running. Open Command Prompt or Git Bash in the project directory.

Step 2: Check Docker installation

docker --version

docker compose version

These commands verify that Docker and Docker Compose are available.

Step 3: Validate the Compose file

docker compose config

This checks the Compose configuration and required environment variables. Ensure the .env file has been created first.

Step 4: Build and start the containers

docker compose up --build -d

This builds the application image and starts the application and database containers in the background.

Step 5: Check running containers

docker compose ps

The application and database services should be running. The database should become healthy before the application starts.

Step 6: View application logs

docker compose logs app

docker compose logs db

These commands help identify application startup errors and database connection problems.

Step 7: Access the application

Open a browser and enter:

http://localhost:5000

The Flask application should respond if the application has been implemented and started correctly.

10.2 Testing Procedure

Testing verifies that the application works correctly inside the Docker environment.

Test case	Test procedure	Expected result
Container startup	Run docker compose up --build -d.	Both services start successfully.
Application access	Open http://localhost:5000.	The application page loads.
Database health	Run docker compose ps.	The database reports healthy.
Expense entry	Enter a valid expense through the implemented form or API.	The expense is stored successfully.
Budget calculation	Record expenses against a configured budget.	The remaining balance is calculated correctly.
Alert generation	Exceed the configured spending threshold.	The application generates an alert.
Financial report	Request a financial summary.	The report displays the stored data correctly.
Data persistence	Restart the containers and check stored records.	Database records remain available.

The expense, budget, alert, and report tests require the corresponding application features to be implemented. Docker container startup alone does not prove that these business functions work correctly.

10.3 Useful Testing Commands

Check the service status:

```
docker compose ps
```

Display recent logs:

```
docker compose logs --tail=100
```

Follow application logs:

```
docker compose logs -f app
```

Stop the services:

```
docker compose down
```

Restart the services:

```
docker compose up -d
```

The named database volume remains available after docker compose down unless it is explicitly removed.

10.4 Screenshots to Include in the Report

Take genuine screenshots of the following steps after running the project:

1. Project folder structure in File Explorer.
2. Git repository initialization and the git status command.
3. Git branches and commit history.
4. Dockerfile and Docker Compose configuration.
5. Successful image build in the terminal.
6. Running application and database containers using docker compose ps.
7. Browser displaying the application at localhost:5000.
8. Expense entry, budget alert, and financial report tests, if implemented.

Each screenshot should have a caption explaining what it demonstrates. Do not use example output as evidence of a test that has not actually been performed.

10.5 Expected Results

When the configuration and application are implemented correctly, Docker Compose should build the Flask application image, start the application and MySQL containers, and connect the services through the internal Docker network.

The application should become accessible through the browser, while financial records should remain stored in the persistent database volume. Successful execution should be confirmed using container status, application logs, browser testing, and functional test results.

11. Benefits of Git and Docker

11.1 Collaboration

Git allows multiple developers to work on separate branches and merge their changes. GitHub pull requests support review and discussion before changes are integrated.

Docker Compose allows developers to use a common application and database configuration, reducing differences between development environments.

11.2 Version Management

Git maintains a history of source code changes. Developers can inspect commits, identify defects, compare versions, and restore earlier code when necessary.

Meaningful commit messages make it easier to understand the development history.

11.3 Portability

Docker packages the application and its dependencies into an image. The same image can run in compatible Docker environments without manually reinstalling all application dependencies.

Docker Compose also documents the required services and their configuration.

11.4 Deployment

Docker Compose starts the application and database using a single command. This simplifies development deployment and reduces manual setup steps.

The application image can also be published to a container registry for deployment on another machine or compatible hosting environment.

11.5 Scalability

Git branches help developers introduce and test new features without directly modifying the stable version.

Docker supports running additional application instances when demand increases. However, scaling requires suitable load balancing, application configuration, database capacity, and coordination of shared state.

11.6 Maintainability

Git provides traceability and supports collaborative code review. Dockerfiles and Compose configurations document the runtime environment.

Together, these tools make the project easier to update, test, share, and deploy.

11.7 Summary of Benefits

Area	Contribution
Collaboration	Supports independent development and code review.
Version management	Tracks changes and maintains history.
Portability	Provides consistent container environments.
Deployment	Automates service startup and configuration.
Scalability	Supports additional application instances when configured appropriately.
Maintainability	Improves traceability and environment reproducibility.

12. Challenges and Improvements

12.1 Challenges Encountered

1. Git Merge Conflicts

Different developers may modify the same lines of code, creating merge conflicts.

Solution: Use small commits, feature branches, regular synchronization, and code reviews. Resolve conflicts carefully and test the merged application.

2. Docker Configuration Errors

Incorrect file names, missing dependencies, or invalid Dockerfile instructions can prevent an image from building.

Solution: Verify filenames, review build logs, validate the Compose file, and rebuild after correcting the errors.

3. Database Connection Problems

The application may fail to connect to MySQL if the hostname, credentials, or startup sequence is incorrect.

Solution: Use the Compose service name as the hostname, configure credentials through environment variables, add a health check, and implement connection error handling.

4. Data Persistence Issues

Database records can be lost if the database container is removed without persistent storage.

Solution: Use a named Docker volume and maintain regular database backups.

5. Security Risks

Committing passwords to GitHub or exposing the database unnecessarily can compromise financial information.

Solution: Exclude .env from Git, use strong credentials, restrict database exposure, validate user input, and protect financial records with authentication and authorization.

6. Testing Difficulties

The application may start successfully while individual functions such as budget calculations or alert generation remain incorrect.

Solution: Use unit tests, API tests, database integration tests, and manual browser testing to verify both infrastructure and application behavior.

12.2 Future Enhancements

The following improvements can strengthen the project.

- Automated Testing: Add unit and integration tests for expense calculations, budget limits, and reports.
- Continuous Integration: Configure GitHub Actions to run tests whenever changes are pushed.
- Container Registry: Publish validated application images to Docker Hub or GitHub Container Registry.
- Secure Deployment: Use HTTPS, managed secrets, secure database access, and appropriate access controls.
- Monitoring: Add health checks, centralized logs, and application monitoring.
- Automated Backups: Schedule database backups and regularly test restoration.
- AI-Based Expense Analysis: Introduce spending predictions and automatic expense categorization.
- Mobile Access: Develop a mobile interface that uses the existing backend APIs.

12.3 Conclusion

The Personal Expense and Budget Management System demonstrates how Git and Docker can be applied to a practical software engineering project. Git supports version control, branching, collaboration, and commit history, while Docker and Docker Compose provide a consistent environment for running the Flask application and MySQL database.

The proposed implementation includes a structured repository, a suitable branching workflow, a Dockerfile, a Compose configuration, and a testing procedure. These practices improve portability, maintainability, and deployment consistency.

Future enhancements such as automated testing, continuous integration, secure deployment, and advanced financial analysis can make the system more reliable and suitable for wider use.

Additional implementation files required

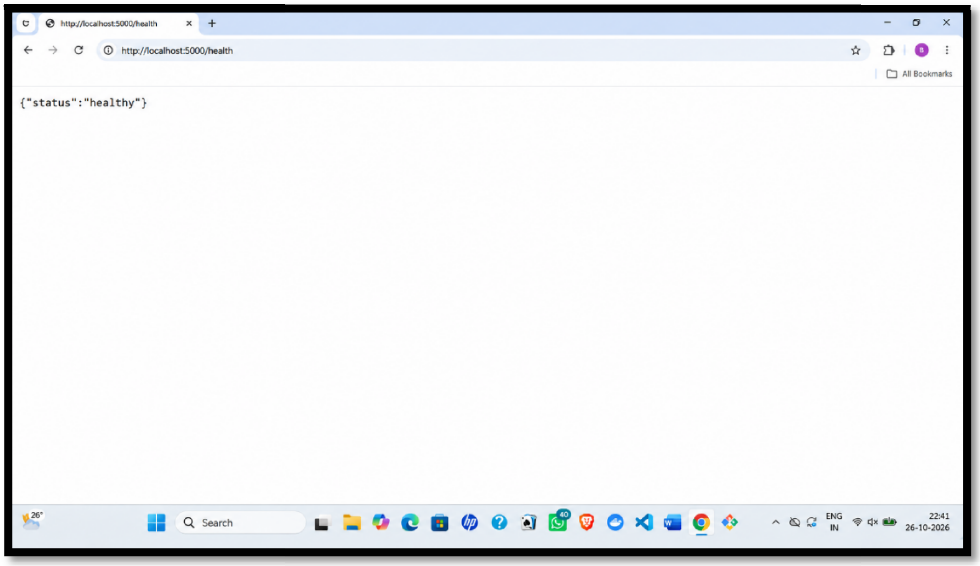
To make the Docker deployment work, include these files in your project.

A. app.py — Basic Flask Application

```
from flask import Flask, request, jsonify
import mysql.connector
import os
app = Flask(__name__)
def get_db():
    return mysql.connector.connect(
        host=os.getenv("DB_HOST", "db"),
        user=os.getenv("DB_USER", "expense_user"),
        password=os.environ["DB_PASSWORD"],
        database=os.getenv("DB_NAME", "expense_db")
    )
@app.route("/")
def home():
    return """
<h1>Personal Expense and Budget Management System</h1>
<p>Application is running successfully!</p>
<p>Use /health to check database connectivity.</p>
    """
@app.route("/health")
def health():
    try:
db = get_db()
db.close()
        return jsonify({"status": "healthy"})
    except Exception:
        return jsonify({"status": "database unavailable"}), 503
@app.route("/expenses", methods=["GET", "POST"])
def expenses():
    if request.method == "POST":
        data = request.get_json(silent=True) or {}
        try:
            amount = float(data["amount"])
```

```
            category = str(data["category"]).strip()
            date = data["date"]
user_id = int(data["user_id"])
            if amount <= 0 or not category:
                return jsonify({"error": "Invalid expense"}), 400
db = get_db()
            cursor = db.cursor()
cursor.execute(
                """INSERT INTO expenses
                  (user_id, amount, category, expense_date)
                  VALUES (%s, %s, %s, %s)""",
                (user_id, amount, category, date)
            )
db.commit()
cursor.close()
db.close()
            return jsonify({"message": "Expense added"}), 201
        except (KeyError, ValueError):
            return jsonify({"error": "Invalid input"}), 400
        except Exception:
            return jsonify({"error": "Unable to save expense"}), 500
db = get_db()
    cursor = db.cursor(dictionary=True)
cursor.execute(
        """SELECT expense_id, user_id, amount, category, expense_date
          FROM expenses ORDER BY expense_date DESC"""
    )
    rows = cursor.fetchall()
cursor.close()
db.close()
    return jsonify(rows)
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

This is a basic demonstration backend with a homepage, database health check, and expense API. A complete submission still needs authentication, user-specific authorization, budget calculations, alerts, reports, and a suitable frontend.



B. Verify the deployment

After creating the files and configuration described above, run:

docker compose up --build -d

docker compose ps

Test the homepage in your browser at http://localhost:5000.

Check database connectivity at http://localhost:5000/health. If the database is ready and the credentials are correct, the expected response is:

{"status": "healthy"}